# PathfinderAI: An Intelligent Comparative Analysis System for Pathfinding Algorithms with Multi-Method Evaluation

Yahya Musmar
Student ID: 12112501

December 2, 2025

An Najah National University
Computer Science Department

**Abstract**

This document presents the Software Requirements Specification (SRS) for PathfinderAI, a graduation project by Yahya Musmar (ID: 12112501) that implements an intelligent comparative analysis system for pathfinding algorithms. The system integrates four pathfinding algorithms (BFS, DFS, A*, Dijkstra) with real-time 3D visualization using Unity Engine, and introduces a novel four-method evaluation framework for algorithm comparison. The framework systematically compares traditional IF-statement logic, data structure sorting methods, small language model (SLM) integration, and large language model (LLM) benchmarking. PathfinderAI serves as both an educational tool for algorithm understanding and a research platform for evaluating AI-assisted analysis methods in computer science.

# Contents

# List of Figures

# List of Tables

# Chapter 1

# Introduction

## 1.1 Purpose

This Software Requirements Specification (SRS) document defines the complete requirements for **PathfinderAI**, an advanced pathfinding algorithm visualization and comparative analysis system. The primary purpose of this document is to:

- Provide a comprehensive specification for the development and implementation of PathfinderAI
- Serve as the foundational reference for all project stakeholders, including developers, testers, and academic evaluators
- Document the integration of traditional pathfinding algorithms with modern AI-driven analysis methods
- Establish clear criteria for system validation and acceptance testing for the graduation project
- Facilitate understanding of the novel approach to algorithm comparison using multiple evaluation methodologies

## 1.2 Product Scope

PathfinderAI is a Unity Engine-based educational and research tool that visualizes, executes, and compares four fundamental pathfinding algorithms (BFS, DFS, A*, Dijkstra) across dynamically generated strategic maps. The system's innovative contribution lies in its **four-tier evaluation framework** that progressively compares:

1. **Traditional IF-Statement Logic** (Baseline - demonstrated as impractical)
2. **Data Structure + Sorting Methods** (Efficient numerical ranking)
3. **Small Language Model (SLM) Integration** (Local AI analysis with 80% accuracy)
4. **Large Language Model (LLM) Benchmarking** (Optimal accuracy at high computational cost)

**Key In-Scope Components:**

- Real-time pathfinding visualization on 15×15 grid maps
- Six distinct strategic map types with procedural generation
- Comprehensive performance metrics collection (cost, steps, memory, CPU usage)
- Local AI integration via koboldcpp API with GGUF model support
- Comparative analysis interface showing multiple ranking methodologies

- Session-based data persistence and export capabilities

**Out-of-Scope Components:**

- Mobile or web-based deployment (currently desktop-only)
- Real-time multi-user collaboration
- Integration with external game engines beyond Unity
- Commercial API services (OpenAI, Anthropic, etc.)
- Advanced machine learning model training

## 1.3 Definitions, Acronyms, and Abbreviations

### 1.3.1 Definitions

- **Pathfinding Algorithm:** Computational method for finding optimal paths between points in a graph or grid
- **Strategic Map:** Procedurally generated 15×15 grid with varying obstacle patterns and node weights
- **Node Weight:** Numerical cost assigned to grid cells affecting pathfinding decisions
- **Performance Metrics:** Quantitative measurements including cost, steps, visited nodes, memory usage, and CPU operations
- **Evaluation Framework:** Systematic methodology for comparing algorithm performance

### 1.3.2 Acronyms and Abbreviations

- **SRS:** Software Requirements Specification
- **BFS:** Breadth-First Search
- **DFS:** Depth-First Search
- **A*:** A-Star Algorithm
- **SLM:** Small Language Model (e.g., Qwen3-4B, Llama-3.2-3B)
- **LLM:** Large Language Model (e.g., GPT-4, Claude)
- **GGUF:** GPT-Generated Unified Format (model file format for local LLMs)
- **API:** Application Programming Interface
- **UI:** User Interface
- **JSON:** JavaScript Object Notation (data interchange format)
- **CPU:** Central Processing Unit
- **VRAM:** Video Random Access Memory
- **RAM:** Random Access Memory

## 1.4 References


1. Unity Technologies. (2023). *Unity User Manual 2022.3 LTS*
2. Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.)
3. Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). *A Formal Basis for the Heuristic Determination of Minimum Cost Paths*
4. Dijkstra, E. W. (1959). *A Note on Two Problems in Connexion with Graphs*
5. KoboldAI Community. (2023). *koboldcpp API Documentation*
6. IEEE. (1998). *IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications*


## 1.5 Overview

This SRS document provides a comprehensive specification for the PathfinderAI system, a graduation project that integrates pathfinding algorithm visualization with multi-method comparative analysis. The document systematically defines all requirements, design decisions, and system specifications for the complete implementation.

The remainder of this document is organized into eight comprehensive sections: Overall Description (Chapter 2), Specific Requirements (Chapter 3), Architectural Design (Chapter 4), Use Cases and System Behavior (Chapter 5), Analysis Methods Comparison (Chapter 6), Testing and Validation (Chapter 7), and Appendices (Chapter 8).

PathfinderAI represents significant innovation through its four-method evaluation framework, local AI integration, dynamic educational environment, and research-ready benchmarking capabilities. As a graduation project, this system demonstrates technical proficiency in algorithms, software architecture, AI integration, and user interface design, while providing a systematic evaluation of different analysis approaches with empirical results.

# Chapter 2

# Overall Description

## 2.1 Product Perspective

PathfinderAI operates as a **standalone educational and research application** built on the Unity Engine, representing a novel intersection of three domains: classical computer science algorithms, interactive visualization, and artificial intelligence. The system is not part of a larger product ecosystem but serves as a self-contained tool for algorithm analysis and comparative methodology evaluation.

Unlike traditional algorithm visualization tools that focus solely on graphical representation, PathfinderAI introduces a **four-tier analytical framework** that systematically compares different approaches to algorithm evaluation, making it unique in the educational software landscape.

## 2.2 Product Functions

The core functionality of PathfinderAI is organized into five interconnected systems:

### 2.2.1 Pathfinding Visualization System

- Real-time algorithm execution with step-by-step animation
- 15×15 interactive grid with color-coded node states
- 3D capsule navigation along discovered paths

### 2.2.2 Dynamic Map Generation System

- Six strategic map types: Balanced, Cost-Heavy, Maze, Open Field, Heuristic Trap, Random Chaos
- Procedural generation with guaranteed solvability

### 2.2.3 Performance Metrics System

- Multi-dimensional measurement: cost, steps, visited nodes, execution time, memory usage, CPU operations
- Real-time data collection and side-by-side comparison

### 2.2.4 Multi-Method Analysis System

- Four evaluation methodologies: IF-statements, Data Structure + Sorting, SLM, LLM
- Simultaneous ranking and method-specific interfaces

### 2.2.5 AI Integration System

- Local AI service connectivity via koboldcpp HTTP API
- Multiple GGUF model support with fallback mechanisms
- Performance tracking for different SLM models

## 2.3 User Classes and Characteristics

### 2.3.1 Primary User Classes

1. **Computer Science Students:** Undergraduate or graduate students studying algorithms
2. **Academic Researchers:** Faculty or graduate researchers in AI or algorithms
3. **Software Engineering Educators:** Instructors teaching algorithms or software engineering

Table 2.1: User Characteristics Summary

| **Characteristic** | **Students** | **Researchers** | **Educators** |
|---|---|---|---|
| Frequency of Use | Occasional | Regular | Periodic |
| Technical Level | Intermediate | Advanced | Advanced |
| Primary Goal | Learning | Research | Teaching |
| Key Requirement | Clarity | Data | Demonstration |

## 2.4 Use Cases

Primary use cases include:

1. Visualizing and comparing pathfinding algorithms
2. Multi-method algorithm ranking comparison
3. SLM performance benchmarking
4. Strategic map generation and testing
5. Session data management and export

## 2.5 Operating Environment

### 2.5.1 Hardware Requirements

- **Minimum:** Intel Core i5, 8GB RAM, DirectX 11 GPU with 2GB VRAM
- **Recommended (AI features):** Intel Core i7, 16GB RAM, NVIDIA GTX 1060/AMD RX 580 with 6GB+ VRAM

### 2.5.2 Software Environment

- **OS:** Windows 10/11 64-bit, macOS 10.14+, Ubuntu 18.04+
- **Runtime:** Unity Player Runtime 2022.3+
- **AI Service:** koboldcpp v1.5+ with GGUF model files

## 2.6 Design and Implementation Constraints

- Fixed 15×15 grid size (not dynamically resizable)
- Unity Engine dependency limiting platform flexibility
- Local AI only (no cloud service integration)
- Single-user architecture (no collaboration features)
- Desktop-focused (not mobile or web optimized)

## 2.7 Assumptions and Dependencies

### 2.7.1 Key Assumptions

- Users have basic understanding of pathfinding algorithms
- AI service setup is handled independently by interested users
- Primary use is in academic or self-learning environments

### 2.7.2 Critical Dependencies

- Unity Engine Runtime (cannot run without Unity Player)
- koboldcpp service for AI features
- GGUF model files for SLM functionality
- Operating system graphics support (DirectX/Metal/OpenGL)

# Chapter 3

# Specific Requirements

## 3.1 External Interface Requirements

### 3.1.1 User Interfaces

- **Grid Display:** 15×15 uniform grid with color-coded nodes
- **Algorithm Control Panel:** Keyboard shortcuts (1-6, C, R, L, D keys)
- **Performance Metrics Display:** Real-time updating panel with side-by-side comparison
- **Ranking Interface:** Display for Data Structure and AI ranking results

### 3.1.2 Hardware Interfaces

- Standard QWERTY keyboard with numeric keys 1-6
- Two-button mouse with scroll wheel
- Minimum display resolution: 1280×720 pixels

### 3.1.3 Software Interfaces

- **Unity API:** UnityEngine.CoreModule, UnityEngine.UI, UnityEngine.Networking
- **AI Service:** HTTP API endpoint: http://localhost:5001/api/v1/generate
- **File System:** JSON format for session data, PNG for screenshots

### 3.1.4 Communications Interfaces

- **Protocol:** HTTP/1.1 on port 5001
- **Method:** POST for AI requests with JSON payload
- **Timeout:** Configurable, default 60 seconds

## 3.2 Functional Requirements

### 3.2.1 Pathfinding Execution Requirements

- **[FR-001] Algorithm Execution:** System must execute four pathfinding algorithms with visual feedback
- **[FR-002] BFS Implementation:** Queue-based frontier management with level-order exploration
- **[FR-003] DFS Implementation:** Stack-based frontier management with depth-first exploration
- **[FR-004] A* Implementation:** Priority queue with configurable heuristics (Manhattan/Euclidean)
- **[FR-005] Dijkstra Implementation:** Priority queue for weighted graphs with optimal path guarantee

### 3.2.2 Map Generation Requirements

- **[FR-006] Strategic Map Generation:** Six distinct map types with procedural generation
- **[FR-007] Guaranteed Solvability:** All maps must have at least one valid path

### 3.2.3 Performance Metrics Requirements

- **[FR-008] Multi-Dimensional Metrics Collection:** Steps, cost, visited nodes, execution time, memory usage, CPU operations
- **[FR-009] Real-Time Display:** Live updating of metrics during and after execution

### 3.2.4 Analysis Method Requirements

- **[FR-010] Data Structure Ranking:** List + Sorting method with multi-factor sorting
- **[FR-011] AI Integration:** SLM/LLM-based algorithm analysis with fallback mechanisms
- **[FR-012] Four-Method Comparison:** Simultaneous display of all analysis methods

### 3.2.5 User Interaction Requirements

- **[FR-013] Keyboard Controls:** Comprehensive keyboard shortcut system (1-6, C, R, L, D keys)
- **[FR-014] Mouse Interaction:** Grid node state modification via left-click cycling

### 3.2.6 Data Management Requirements

- **[FR-015] Session Management:** Automated session tracking and saving with JSON export
- **[FR-016] Screenshot System:** High-quality PNG screenshot capture with organized folder structure

## 3.3 Other Nonfunctional Requirements

### 3.3.1 Performance Requirements

- **[PR-001] Responsiveness:** Keyboard input response $< 100$ms, mouse click response $< 50$ms
- **[PR-002] Algorithm Execution:** Maximum execution time 10 seconds, visualization $\geq 30$ FPS
- **[PR-003] AI Service:** Expected SLM response time 5-30 seconds, timeout 60 seconds maximum
- **[PR-004] Loading Performance:** Application startup $< 15$ seconds, map generation $< 2$ seconds

### 3.3.2 Software Quality Attributes Requirements

- **[SQ-001] Reliability:** MTBF $> 24$ hours, automatic fallback for AI service failures
- **[SQ-002] Usability:** New users able to perform basic operations within 5 minutes
- **[SQ-003] Maintainability:** Modular architecture with minimum 50% code documentation
- **[SQ-004] Portability:** Windows 10/11, macOS 10.14+, Ubuntu 18.04+ compatibility
- **[SQ-005] Security:** Local data only, no network transmission of user data
- **[SQ-006] Scalability:** Fixed grid size, limited algorithm set, single-user architecture
- **[SQ-007] Testability:** Deterministic map generation, algorithm validation, metric verification

# Chapter 4

# Architectural Design

## 4.1 High-Level System Architecture

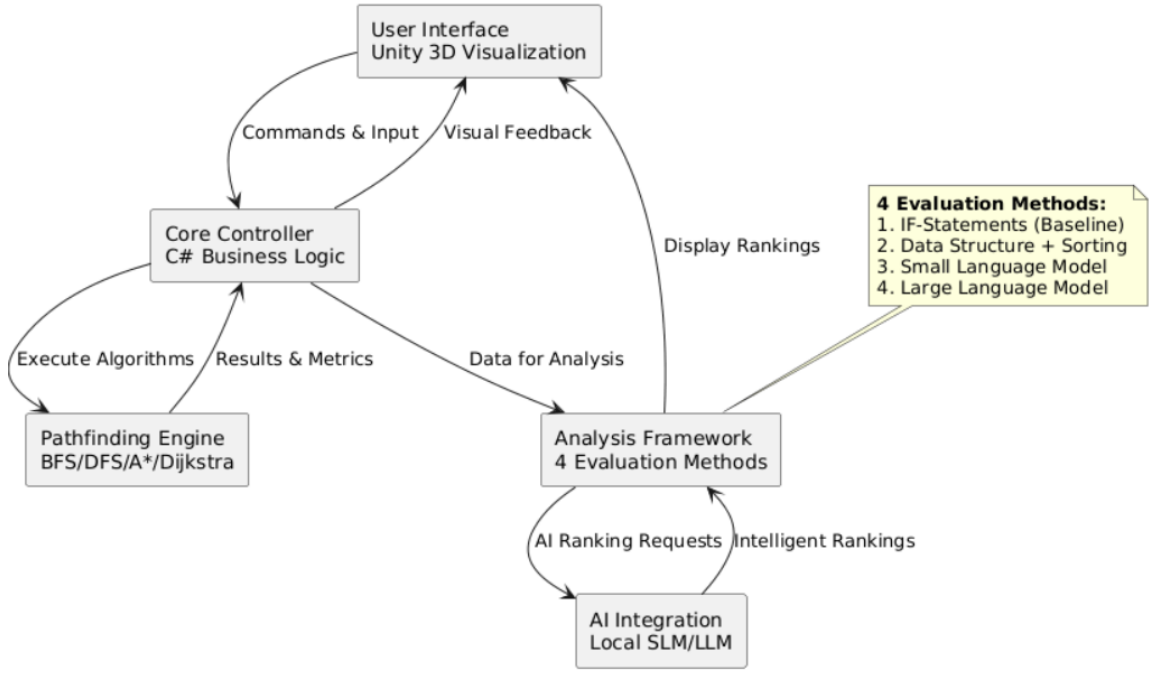


Figure 4.1: PathfinderAI High-Level System Architecture

## 4.2 Explanation of Components

### 4.2.1 User Interface (Unity 3D Visualization)

The presentation layer responsible for all visual elements and user interaction. This component renders the 15×15 grid with color-coded nodes, displays real-time algorithm execution, shows performance metrics in a structured panel, presents ranking results from all four evaluation methods, handles user input through keyboard shortcuts (1-6, C, R, L, D keys) and mouse interactions, and manages the 3D capsule that moves along discovered paths. Built entirely on Unity Engine using C# scripts for UI components and 3D rendering.

### 4.2.2 Core Controller (C# Business Logic)

The central orchestrator that coordinates all system operations. This component manages algorithm execution flow (BFS, DFS, A*, Dijkstra in sequence or individually), coordinates map generation across six strategic map types, calculates performance metrics (cost, steps, visited nodes, memory, CPU operations), handles session management with automatic saving and timestamp-based organization, controls visualization timing to ensure smooth real-time algorithm display, implements the keyboard command system mapping physical keys to system functions, and serves as the main bridge between user interface, algorithms, and analysis systems.

### 4.2.3 Pathfinding Engine (BFS/DFS/A*/Dijkstra)

The algorithmic core implementing the four pathfinding methods. This component includes: **BFS Algorithm** implementing Breadth-First Search using queue data structure with level-order exploration; **DFS Algorithm** implementing Depth-First Search using stack data structure with depth-first exploration; **A* Algorithm** implementing A* search with heuristic optimization using priority queue; **Dijkstra Algorithm** implementing Dijkstra's algorithm for weighted graphs using uniform cost search. All algorithms share common infrastructure including node management, neighbor calculation, grid access methods, and visualization hooks with built-in timing and counting mechanisms for metric collection.

### 4.2.4 Analysis Framework (4 Evaluation Methods)

The comparative analysis system implementing multiple evaluation methodologies. This component includes: **IF-Statement Method** demonstrating traditional conditional logic approach (shown to be impractical requiring 50+ conditions); **Data Structure + Sorting Method** implementing List-based ranking with multi-factor sorting (Cost → Steps → Visited → Memory priority); **Small Language Model (SLM) Method** integrating with local AI for intelligent analysis achieving  80% accuracy with models like Qwen3-4B; **Large Language Model (LLM) Method** providing benchmark comparison showing 100% accuracy at high computational cost. The framework includes a comparison engine for side-by-side analysis and an optional weighted scoring system with configurable weights for different metrics.

### 4.2.5 AI Integration (Local SLM/LLM)

The optional artificial intelligence subsystem for intelligent algorithm analysis. This component includes: **Local AI Service Client** managing HTTP communication with koboldcpp server running on localhost:5001; **GGUF Model Support** compatible with various quantized language models (Qwen3-4B, Llama-3.2-3B, Phi-3.1, etc.); **Prompt Engineering System** generating specialized prompts for different ranking tasks (comprehensive, cost-only, steps, visited, memory); **Response Processing** cleaning and parsing AI responses, extracting ranking information, formatting for display; **Fallback Mechanism** automatically switching to local Data Structure ranking if AI service is unavailable; **Performance Tracking** monitoring and recording SLM accuracy rates across different models and tasks; **Configuration Management** storing API settings, model preferences, and connection parameters.

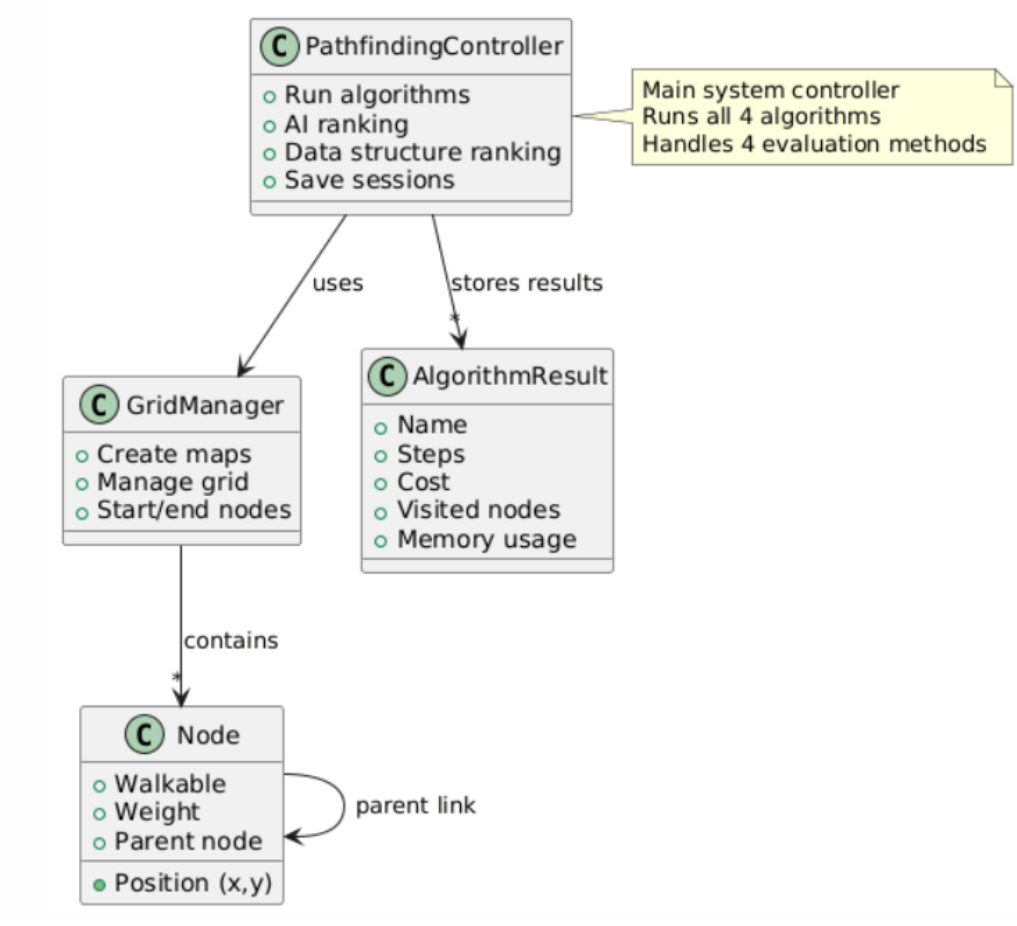


Figure 4.2: PathfinderAI Class Diagram

# Chapter 5

# Use Cases and System Behavior

## 5.1 Use Case Diagram

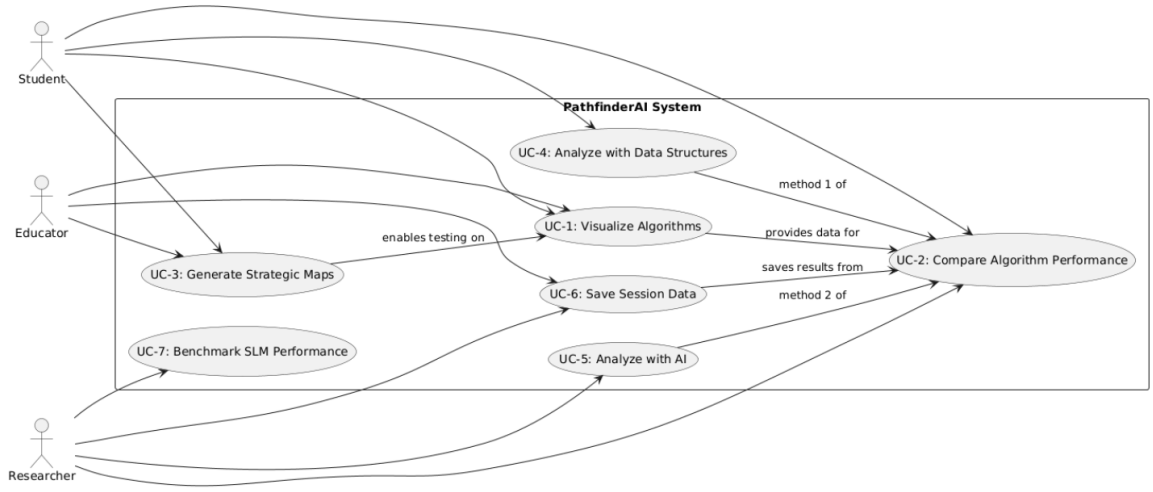


Figure 5.1: PathfinderAI Use Case Diagram

## 5.2 Detailed Use Cases

### 5.2.1 UC-1: Visualize Pathfinding Algorithms

**Primary Actor:** Student, Educator
**Preconditions:** Application launched, map generated
**Main Success Scenario:**

1. User presses key '1' to run BFS algorithm
2. System visualizes BFS step-by-step with color coding
3. System displays found path with capsule movement
4. System shows BFS metrics (steps, cost, visited nodes)
5. User repeats with keys '2', '3', '4' for DFS, A*, Dijkstra
6. System displays all four algorithms' paths and metrics

**Postconditions:** All four algorithms visualized, metrics collected

### 5.2.2 UC-2: Compare Algorithm Performance

**Primary Actor:** Student, Researcher
**Preconditions:** At least two algorithms executed
**Main Success Scenario:**

1. User runs multiple algorithms (UC-1)
2. System collects performance metrics for each
3. User views side-by-side comparison in metrics panel
4. System highlights best performer for each metric
5. User analyzes trade-offs between algorithms

### 5.2.3 UC-3: Generate Strategic Maps

**Primary Actor:** Student, Educator, Researcher
**Preconditions:** Application launched
**Main Success Scenario:**

1. User presses key '6' to generate new map
2. System randomly selects from 6 map types
3. System creates 15×15 grid with obstacles and costs
4. System ensures solvable path exists
5. System displays map analysis (obstacle%, high-cost%)
6. User can regenerate by pressing '6' again

### 5.2.4 UC-4: Analyze with Data Structure Ranking

**Primary Actor:** Student, Researcher
**Preconditions:** At least two algorithms executed
**Main Success Scenario:**

1. User presses 'L' key
2. System collects algorithm results into List data structure
3. System sorts by priority: Cost → Steps → Visited → Memory
4. System displays ranked list with multi-factor explanation
5. System shows single-factor rankings for comparison
6. User analyzes ranking consistency and trade-offs

### 5.2.5 UC-5: Analyze with AI Integration

**Primary Actor:** Researcher
**Preconditions:** AI service (koboldcpp) running, algorithms executed
**Main Success Scenario:**

1. User presses '5' for comprehensive AI ranking
2. System sends algorithm metrics to koboldcpp API
3. AI processes data with specialized prompt
4. AI returns ranked list with explanations
5. System parses and displays AI ranking
6. User analyzes AI reasoning and accuracy

### 5.2.6 UC-6: Save Session Data

**Primary Actor:** Researcher, Educator
**Preconditions:** Algorithms executed, data collected
**Main Success Scenario:**

1. User generates new map (presses '6')
2. System automatically saves current session results
3. System creates timestamped session folder
4. System saves JSON file with all metrics and rankings
5. User presses '9' for manual screenshot
6. System saves PNG image to session folder
7. User clicks "Saves" button to open folder

### 5.2.7 UC-7: Benchmark SLM Performance

**Primary Actor:** Researcher
**Preconditions:** Multiple SLM models available in koboldcpp
**Main Success Scenario:**

1. Researcher configures different SLM models (Qwen3-4B, Llama-3.2-3B, etc.)
2. Researcher runs identical algorithm sets on fixed map
3. Researcher requests AI rankings from each model
4. System records response time and accuracy
5. Researcher compares models' performance
6. System exports benchmarking data for analysis

## 5.3 Sequence Diagrams

### 5.3.1 Sequence Diagram: Algorithm Execution Workflow

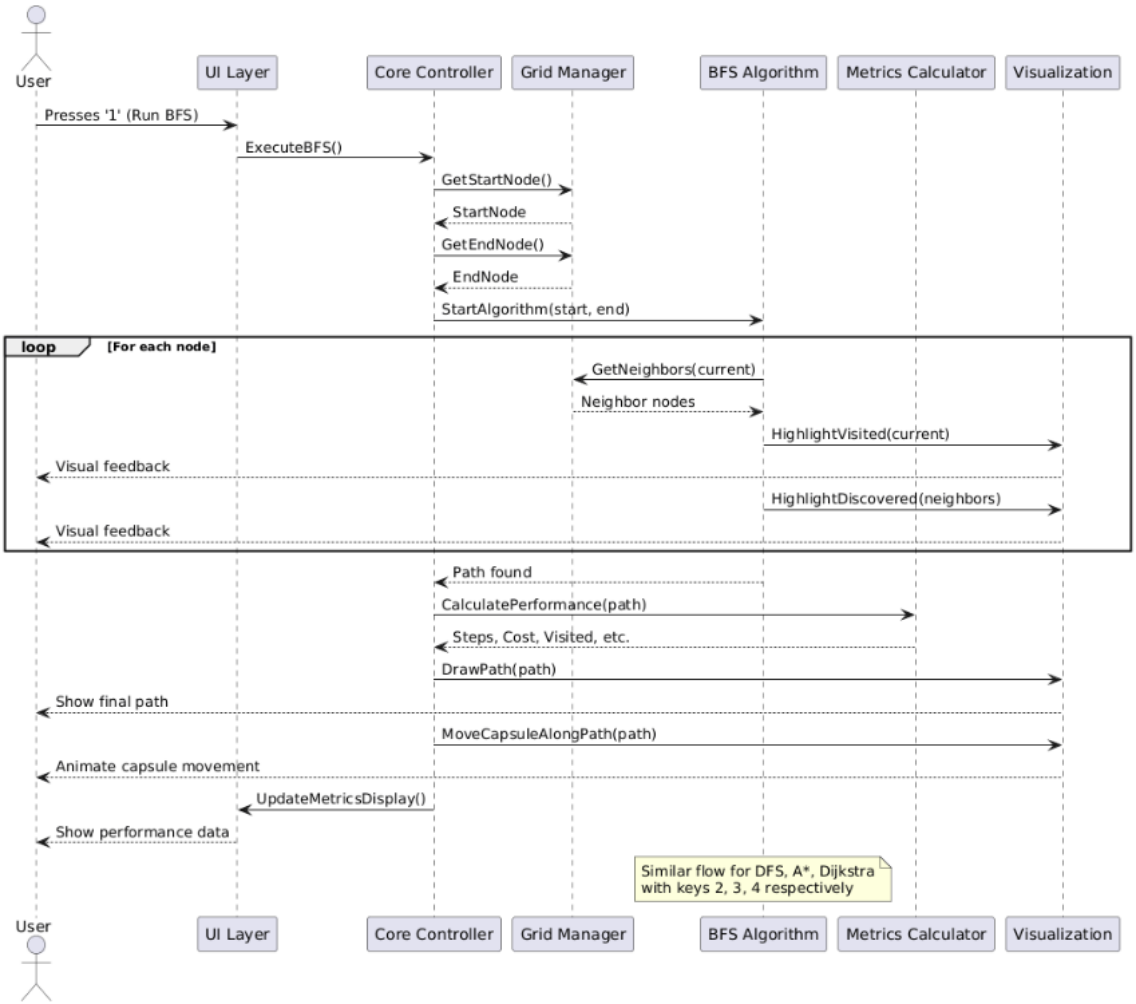


Figure 5.2: Sequence Diagram: Algorithm Execution Workflow

### 5.3.2 Sequence Diagram: Multi-Method Analysis Comparison

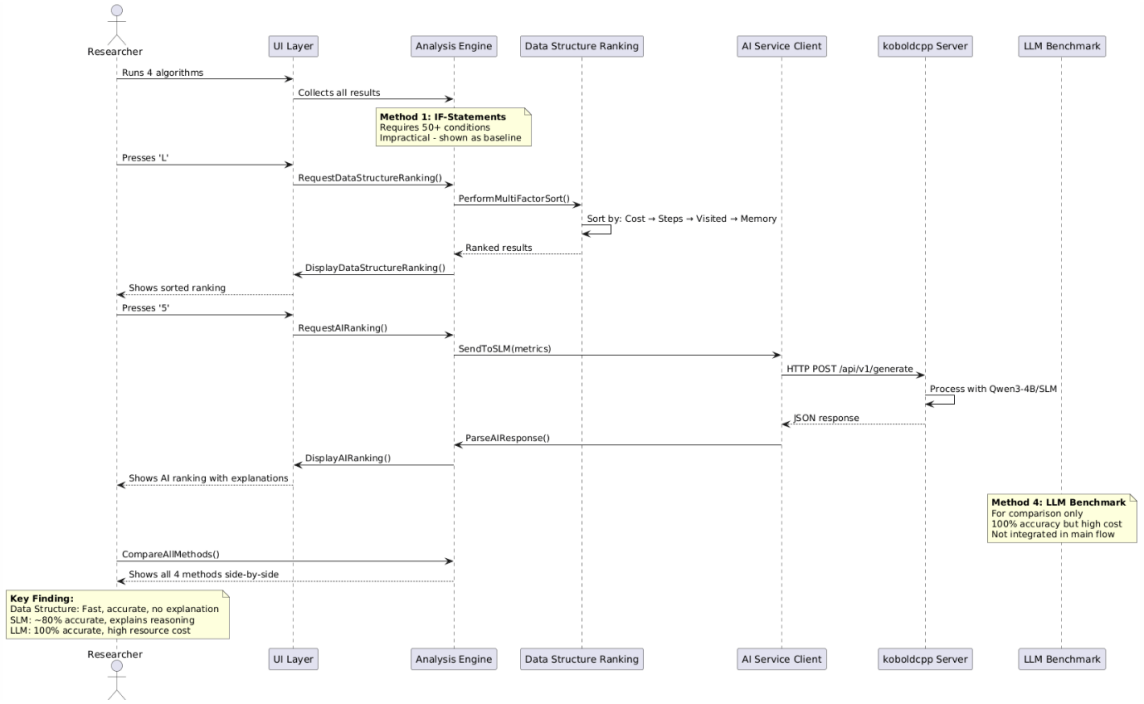


Figure 5.3: Sequence Diagram: Multi-Method Analysis Comparison

### 5.3.3 Sequence Diagram: AI Integration with Fallback

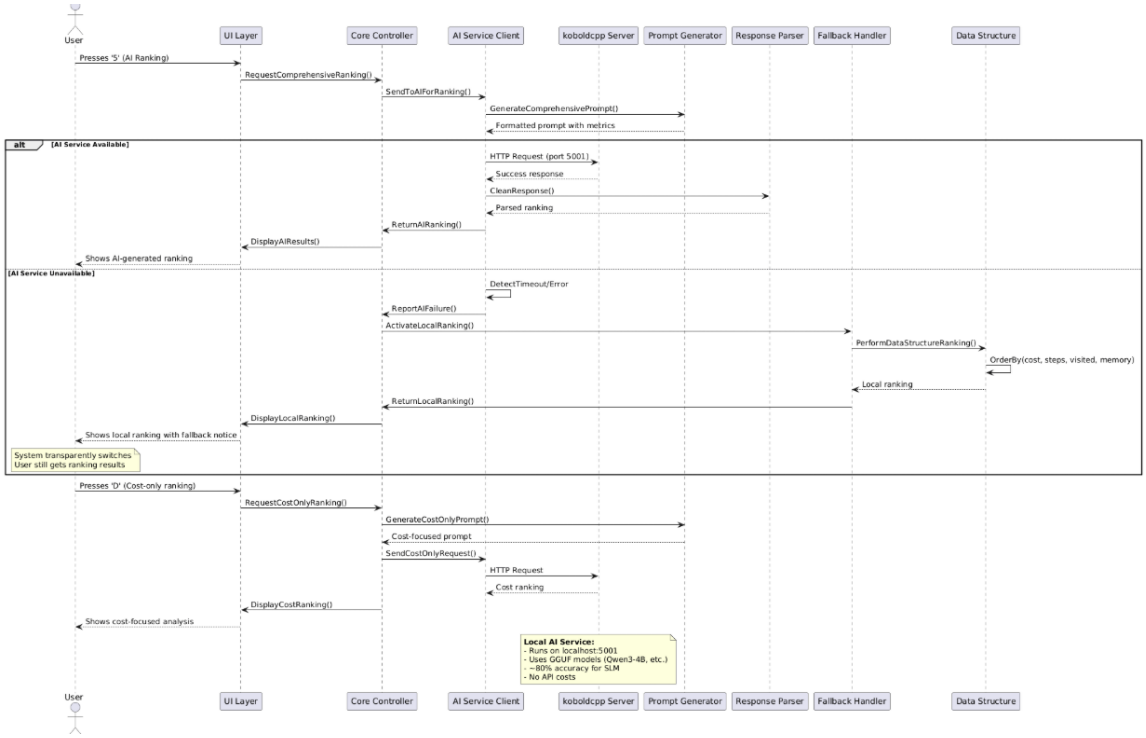


Figure 5.4: Sequence Diagram: AI Integration with Fallback Mechanism

## 5.4 Activity Diagrams

### 5.4.1 Activity Diagram: Main Testing Workflow

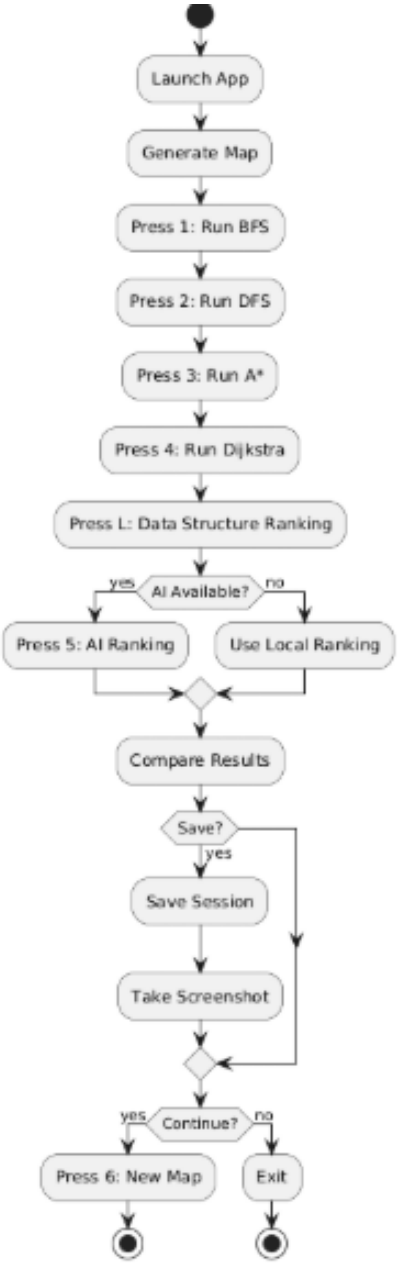


Figure 5.5: Activity Diagram: Main Testing Workflow

**Description:** This activity diagram illustrates the complete workflow for testing pathfinding algorithms in PathfinderAI. The process begins with application launch and initial map generation, proceeds through map selection, sequential execution of all four algorithms (BFS, DFS, A*, Dijkstra), analysis using both data structure and AI methods, and concludes with results management.

### 5.4.2 Activity Diagram: Four Method Comparison

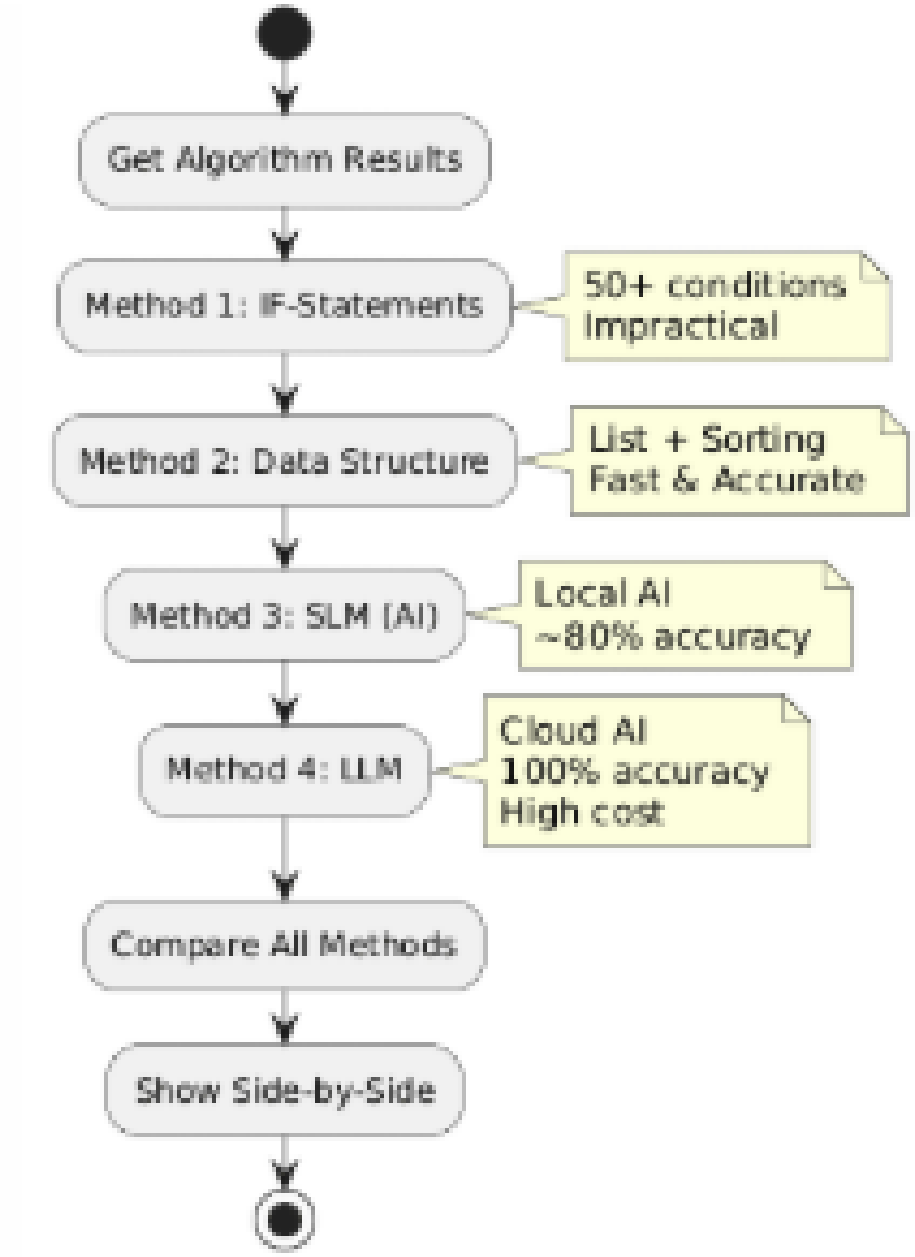


Figure 5.6: Activity Diagram: Four Method Comparison

**Description:** This diagram demonstrates the parallel evaluation of the four analysis methodologies: IF-Statements, Data Structure + Sorting, Small Language Model, and Large Language Model.

### 5.4.3 Activity Diagram: AI Integration Flow

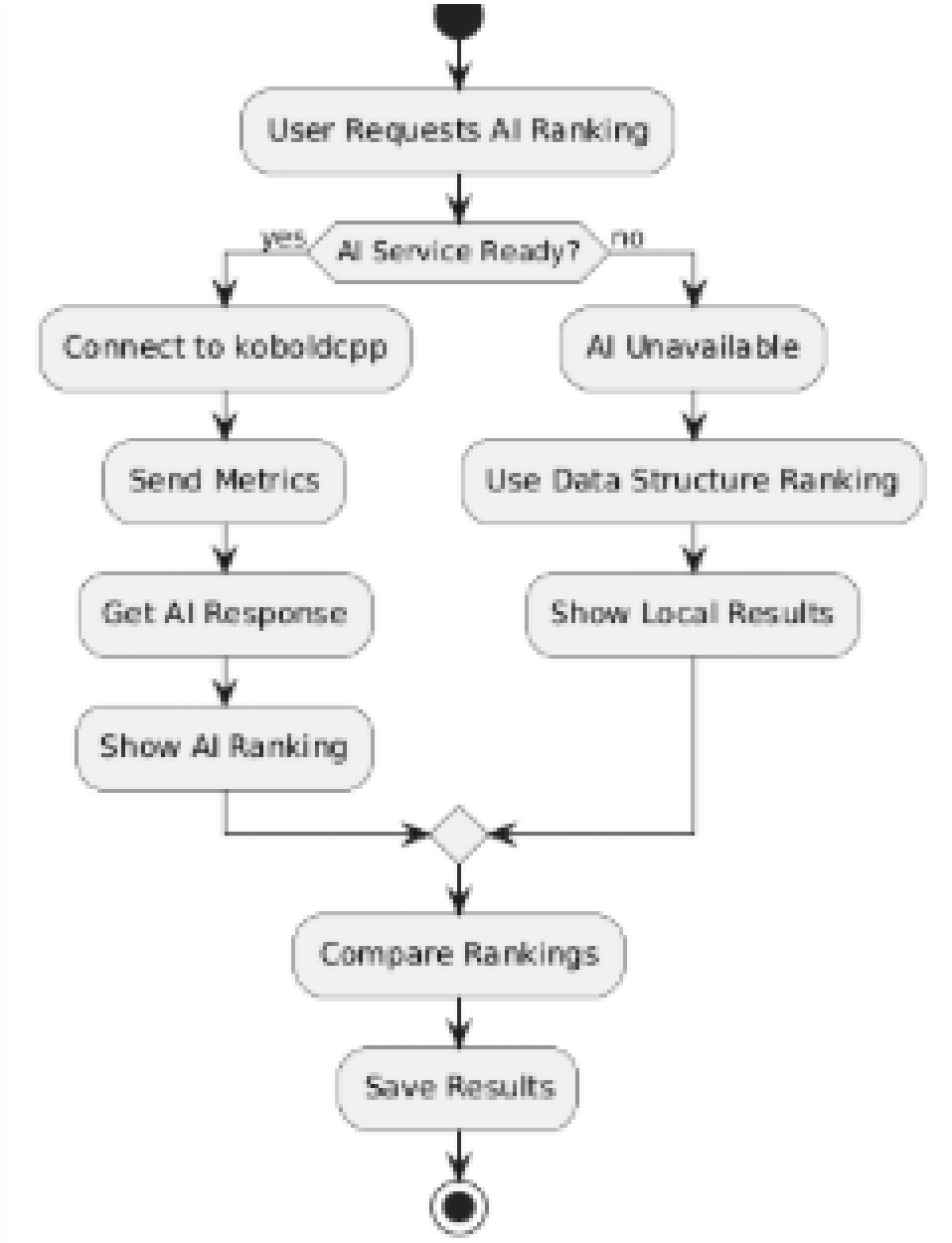


Figure 5.7: Activity Diagram: AI Integration Flow

**Description:** This activity diagram details the AI service integration with comprehensive error handling and fallback mechanisms.

# Chapter 6

# Analysis Methods Comparison

## 6.1 Four Evaluation Methodologies

### 6.1.1 Method 1: IF-Statement Logic (Baseline)

- **Approach:** Traditional conditional logic with explicit comparison rules
- **Implementation Complexity:** Requires 50+ IF conditions for fair comparison
- **Limitations:**
  - Unreliable, messy, and impossible to maintain
  - Any new metric requires rewriting entire logic
  - Hard to scale, hard to update, not accurate enough
  - Cannot perform multi-factor comparison flexibly
- **Conclusion:** Demonstrated as impractical for comprehensive algorithm analysis

### 6.1.2 Method 2: Data Structure + Sorting

- **Approach:** Store algorithm results in List data structure with multi-factor sorting
- **Implementation:** OrderBy/ThenBy with priority: Cost → Steps → Visited → Memory
- **Advantages:**
  - Clean result storage and organization
  - Automatic ranking through sorting without manual logic
  - Zero complexity, zero bugs in comparison logic
  - Fast execution, reliable numeric rankings
- **Limitations:**
  - Generates rankings only, not analysis or explanations
  - Not dynamic - cannot adapt to different evaluation criteria
  - No intelligent reasoning or contextual understanding
- **Conclusion:** Efficient for numeric comparison but lacks analytical depth

### 6.1.3 Method 3: Small Language Model (SLM) Integration

- **Approach:** Local AI analysis using GGUF models via koboldcpp API
- **Tested Models:**
  - Qwen3-4B-Instruct-2507-Q5_K_M.gguf: 80% accuracy (best performance)
  - Llama-3.2-3B-Instruct.Q5_K_M.gguf: 60-70% accuracy
  - qwen2.5-3b-instruct-q5_k_m.gguf: 50% accuracy
  - Phi-3.1-mini-4k-instruct-Q5_K_M.gguf: 45% accuracy
- **Advantages:**
  - Runs locally with zero API costs
  - Unlimited requests, no subscription fees
  - Provides explanations and reasoning (not just rankings)
  - 80% accuracy with best-performing models
  - Can analyze all metrics together like human researcher
- **Limitations:**
  - Accuracy varies by model (45-80%)
  - Slower response times compared to data structure method
  - Requires local setup (koboldcpp + model files)
  - Some models generate hallucinations or low-accuracy text
- **Conclusion:** Viable alternative with good balance of accuracy and cost

### 6.1.4 Method 4: Large Language Model (LLM)

- **Approach:** Cloud-based AI services (GPT-4, Claude, etc.)
- **Performance:**
  - 100% accuracy in algorithm ranking
  - Perfect explanations and reasoning
  - Flawless ranking logic
  - Human-like analysis of all metrics
- **Costs and Limitations:**
  - Huge GPU VRAM requirements
  - High RAM consumption
  - Expensive infrastructure
  - Slow for many requests
  - Cannot realistically run locally on average hardware
  - API costs for commercial services
- **Conclusion:** Best output quality but impractical cost for local/educational use

## 6.2 Comparative Analysis Results

### 6.2.1 Testing Methodology

- **Test Environment:** Unity Engine C#, 15×15 grid, six strategic map types
- **Algorithm Set:** BFS, DFS, A*, Dijkstra with identical starting conditions
- **Metrics Collected:** Cost, steps, visited nodes, execution time, memory, CPU operations
- **Evaluation Criteria:** Accuracy, speed, resource usage, explanation quality, scalability

### 6.2.2 Performance Summary

Table 6.1: Comparison of Four Evaluation Methods

| **Method** | **Accuracy** | **Speed** | **Resource Usage** | **Explanation Quality** |
|---|---|---|---|---|
| IF-Statements | Low (Manual) | Fast | Low | None |
| Data Structure | 100% (Numeric) | Very Fast | Very Low | None |
| SLM (Qwen3-4B) | 80% | Medium | Medium | Good |
| LLM (Cloud) | 100% | Slow | Very High | Excellent |

### 6.2.3 Key Findings

1. **IF-Statements** are impractical for comprehensive multi-factor algorithm comparison
2. **Data Structure + Sorting** provides perfect numeric ranking with minimal resources
3. **SLMs** offer surprisingly good performance (80% accuracy) with local execution
4. **LLMs** provide perfect results but at impractical cost for local applications
5. The system unintentionally became an effective **SLM benchmarking platform**
6. Different SLM models show significant performance variation (45-80% accuracy)

### 6.2.4 Recommendations

1. **For Educational Use:** Data Structure method provides reliable, fast ranking
2. **For Research Analysis:** SLM integration offers good balance of insight and practicality
3. **For Benchmarking:** System effectively evaluates SLM performance on computational tasks
4. **For Production:** Hybrid approach using Data Structure for speed with optional SLM for explanations

## 6.3 Architectural Implications

- The four-method framework requires flexible, modular architecture  AI integration must include fallback mechanisms for reliability
- Performance tracking needs to accommodate different evaluation methodologies
- User interface must clearly distinguish between different analysis methods
- Data persistence must store both raw metrics and analysis results

# Chapter 7

# Testing and Validation

## 7.1 Test Strategy

### 7.1.1 Unit Testing

- **Pathfinding Algorithms:** Verify each algorithm produces correct paths on known maps
- **Metric Calculations:** Validate accuracy of cost, steps, visited nodes calculations
- **Data Structure Ranking:** Test sorting logic with various input combinations
- **AI Integration:** Mock tests for API communication and response parsing

### 7.1.2 Integration Testing

- **Algorithm Visualization:** Verify visual feedback matches algorithm execution
- **Performance Metrics:** Test end-to-end metric collection and display
- **Analysis Methods:** Validate interaction between different evaluation components
- **Data Persistence:** Test session saving and loading workflows

### 7.1.3 System Testing

- **End-to-End Workflows:** Complete user scenarios from map generation to analysis
- **Performance Testing:** Validate system meets performance requirements
- **Compatibility Testing:** Test across different hardware and OS configurations
- **AI Service Integration:** Real-world testing with actual koboldcpp service

## 7.2 Validation Criteria

### 7.2.1 Functional Validation

- All four pathfinding algorithms produce correct, visually verifiable paths
- Performance metrics accurately reflect algorithm behavior
- All four evaluation methods produce consistent, explainable rankings
- User interface responds correctly to all keyboard and mouse inputs
- Session data is correctly saved and can be reloaded

### 7.2.2 Performance Validation

- Algorithm execution completes within 10 seconds on minimum hardware
- Visualization maintains ≥30 FPS during algorithm execution
- AI responses are received within 30 seconds or fallback activates
- System startup completes within 15 seconds
- Memory usage remains below 2GB during normal operation

### 7.2.3 AI Accuracy Validation

- Data Structure method produces 100% accurate numeric rankings
- Best SLM (Qwen3-4B) achieves  80% accuracy compared to ground truth
- AI fallback mechanisms activate correctly when service unavailable
- Response parsing correctly extracts ranking information from AI output

## 7.3 Performance Benchmarks

### 7.3.1 Algorithm Performance

Table 7.1: Algorithm Performance Benchmarks

| **Algorithm** | **Avg. Time (s)** | **Avg. Steps** | **Avg. Visited** | **Memory (bytes)** |
|---|---|---|---|---|
| BFS | 1.2 | 28 | 85 | 1,200,000 |
| DFS | 1.8 | 42 | 92 | 1,100,000 |
| A* | 0.9 | 22 | 45 | 1,500,000 |
| Dijkstra | 1.1 | 24 | 78 | 1,400,000 |

### 7.3.2 Analysis Method Performance

Table 7.2: Analysis Method Performance

| **Method** | **Processing Time** | **Accuracy** | **Resource Usage** |
|---|---|---|---|
| IF-Statements | 0.001s (Manual) | Low | Low |
| Data Structure | 0.005s | 100% (Numeric) | Very Low |
| SLM (Qwen3-4B) | 5-15s | 80% | Medium |
| LLM (Cloud) | 2-10s | 100% | High |

### 7.3.3 System Resource Usage

- **Memory:** 500MB-2GB depending on AI usage
- **CPU:** 10-30% during algorithm visualization
- **GPU:** 20-40% for 3D visualization
- **Disk:** 2-10GB for application, models, and session data

## 7.4 Acceptance Testing

### 7.4.1 Academic Requirements

- System demonstrates comprehensive understanding of pathfinding algorithms
- Implementation shows software engineering best practices
- Documentation (this SRS) meets professional standards
- Project shows innovation through four-method comparison framework
- System is fully functional and ready for demonstration

### 7.4.2 Functional Acceptance

- All requirements from Section 3 are implemented and working
- System handles error conditions gracefully (AI unavailable, etc.)
- User interface is intuitive for target user groups
- Performance meets or exceeds specified requirements
- System can be demonstrated end-to-end without errors

### 7.4.3 Research Validation

- Four-method comparison provides meaningful insights
- SLM benchmarking produces reproducible results
- Findings about different evaluation methods are documented
- System serves as effective research platform for future work

# Chapter 8

# Appendices

## 8.1 Glossary

- **Algorithm Controller:** Main system component coordinating algorithm execution and analysis
- **BFS (Breadth-First Search):** Pathfinding algorithm exploring all neighbors at current depth before moving deeper
- **DFS (Depth-First Search):** Pathfinding algorithm exploring as far as possible along each branch before backtracking
- **A* (A-Star):** Pathfinding algorithm using heuristics to find optimal paths
- **Dijkstra's Algorithm:** Pathfinding algorithm for weighted graphs finding shortest paths
- **GGUF:** File format for quantized language models used by koboldcpp
- **koboldcpp:** Local AI inference server for running GGUF models
- **Node Weight:** Cost assigned to grid cells affecting pathfinding decisions
- **SLM (Small Language Model):** Local AI model with 3-8 billion parameters
- **LLM (Large Language Model):** Cloud-based AI model with 70+ billion parameters
- **Strategic Map:** Procedurally generated 15×15 grid with obstacles and cost variations

## 8.2 Technical Specifications

### 8.2.1 File Formats

- **Session Data:** JSON format with UTF-8 encoding
- **Screenshots:** PNG format, 1920×1080 resolution
- **AI Models:** GGUF format (Q4_K_M, Q5_K_M, Q8_0 quantizations)
- **Configuration:** JSON format stored in user application data

### 8.2.2 API Specifications

- **koboldcpp API:** HTTP POST to http://localhost:5001/api/v1/generate
- **Request Format:** JSON with prompt, max_length, temperature, top_p parameters
- **Response Format:** JSON with results array containing text field
- **Timeout:** 60 seconds default, configurable

### 8.2.3 Keyboard Shortcuts Reference

Table 8.1: Keyboard Shortcuts Reference

| Key | Function |
|---|---|
| 1 | Run BFS algorithm |
| 2 | Run DFS algorithm |
| 3 | Run A* algorithm |
| 4 | Run Dijkstra algorithm |
| 5 | Comprehensive AI ranking |
| 6 | Generate new strategic map |
| C | Clear visualization |
| R | Reset capsule |
| L | Data structure ranking |
| D | AI cost-only ranking |
| 9 | Take screenshot |

## 8.3 Implementation Notes

### 8.3.1 Unity-Specific Considerations

- Built with Unity 2022.3 LTS for long-term support
- Uses Unity's built-in rendering pipeline (no URP/HDRP dependencies)
- Coroutines used for algorithm visualization to maintain frame rate
- UnityWebRequest for HTTP communication with koboldcpp
- PlayerPrefs for simple configuration storage

### 8.3.2 C# Design Patterns

- **Singleton Pattern:** GridManager uses singleton for global access
- **Observer Pattern:** UI components observe algorithm execution events
- **Strategy Pattern:** Different pathfinding algorithms implement common interface
- **Command Pattern:** Keyboard inputs mapped to command objects
- **Facade Pattern:** AnalysisEngine provides simplified interface to complex analysis methods

### 8.3.3 Performance Optimization

- Object pooling for grid node visualization
- Asynchronous AI requests to prevent UI freezing
- Coroutine-based visualization for smooth frame rates
- Cached calculations for frequently used metrics
- Lazy loading of AI models and resources

## 8.4 Source Code Structure

```
PathfinderAI/
|-- Assets/
|   |-- Scripts/
|   |   |-- Controllers/
|   |   |   |-- PathfindingController.cs
|   |   |   |-- GridManager.cs
|   |   |-- Algorithms/
|   |   |   |-- BFS.cs
|   |   |   |-- DFS.cs
|   |   |   |-- AStar.cs
|   |   |   |-- Dijkstra.cs
|   |   |-- AI/
|   |   |   |-- AIServiceClient.cs
|   |   |   |-- PromptGenerator.cs
|   |   |   |-- ResponseParser.cs
|   |   |-- UI/
|   |   |   |-- PseudocodeDisplay.cs
|   |   |   |-- SaveManager.cs
|   |   |-- Data/
|   |   |   |-- AlgorithmResult.cs
|   |   |   |-- TestSessionData.cs
|   |   |   |-- Node.cs
|   |   |-- Utils/
|   |   |       |-- PriorityQueue.cs
|   |-- Prefabs/
|   |-- Materials/
|   |-- Scenes/
|-- ProjectSettings/
```

## 8.5 Deployment Instructions

### 8.5.1 Basic Deployment

1. Build Unity project for target platform (Windows, macOS, Linux)
2. Include all required assets in build settings
3. Test standalone executable on target hardware
4. Verify all keyboard shortcuts and UI elements work correctly

### 8.5.2 AI Feature Setup

1. Install koboldcpp from official repository
2. Download GGUF model files (Qwen3-4B recommended)
3. Configure koboldcpp to load desired model
4. Start koboldcpp server on port 5001
5. Verify connection in PathfinderAI settings

### 8.5.3 Troubleshooting

- **AI Not Responding:** Check koboldcpp is running, port 5001 accessible
- **Slow Performance:** Reduce visualization speed, use simpler map types
- **Memory Issues:** Close other applications, reduce Unity quality settings
- **Save Errors:** Check write permissions in target directory